

Test Report

Car Dealership Inventory System

Prepared by: Sanampreet Singh

Date: July 2026

Project Type: Placement Technical Assessment

*Backend automated integration testing (Jest + Supertest)
and frontend manual end-to-end workflow verification*

Contents

Executive Summary	2
1 Backend: Automated Test Report	3
1.1 Test Execution Summary	3
1.2 Test Suites Overview	3
1.3 Admin Vehicle API Integration Tests	3
1.4 Authentication Endpoint Tests	4
1.5 Public & User Vehicle API Integration Tests	4
1.6 Cloudinary Live Integration Test	4
1.7 Code Coverage Summary	4
1.8 Backend Test Execution Screenshots	5
2 Frontend: Manual Workflow Test Report	6
2.1 Scope and Methodology	6
2.2 Test Case Summary	6
2.3 Detailed Test Cases	7
2.4 Frontend Test Summary	14
3 Conclusion	15

Executive Summary

This report documents the testing performed on the Car Dealership Inventory System, covering both the backend REST API and the frontend single-page application.

- **Backend:** Developed using Test-Driven Development (TDD) with automated integration tests (Jest, Supertest, mongodb-memory-server). **30 of 30 tests passed**, with **87.41%** overall statement coverage.
- **Frontend:** Verified through structured manual end-to-end testing across all core user journeys — authentication, browsing/search, purchasing, and admin inventory management. **12 of 12 manual test cases passed**.

1 Backend: Automated Test Report

1.1 Test Execution Summary

All backend integration tests were executed using:

```
npx jest --coverage
```

Metric	Result
Test Suites	4 passed, 4 total
Tests	30 passed, 30 total
Snapshots	0 total
Execution Time	9.336 s

1.2 Test Suites Overview

Test File	Status
tests/integration/admin.vehicle.test.ts	PASS
tests/integration/auth.test.ts	PASS
tests/integration/user.vehicle.test.ts	PASS
tests/integration/cloudinary.test.ts	PASS

1.3 Admin Vehicle API Integration Tests

POST /api/admin/vehicles

- Should create a vehicle with an image when admin submits valid data (201)
- Should fail if a vehicle with the exact make, model, year, and category already exists (409)
- Should fail if required fields are missing (400)
- Should block a request missing an auth token (401)
- Should block a standard user from creating a vehicle (403)

PUT /api/admin/vehicles/:id

- Should allow an admin to update text fields only (200)
- Should allow an admin to upload a new image and replace the old one (200)
- Should return 404 if updating a non-existent vehicle ID
- Should return 400 for an invalid MongoDB ID format
- Should block a standard user from updating a vehicle (403)

POST /api/admin/vehicles/:id/restock

- Should allow an admin to increase the quantity (200)
- Should fail if addedQuantity is negative or invalid (400)
- Should block a standard user from restocking a vehicle (403)

DELETE /api/admin/vehicles/:id

- Should block a standard user from deleting a vehicle (403)
- Should return 404 if deleting a non-existent vehicle ID
- Should return 400 for an invalid MongoDB ID format
- Should allow an admin to delete a vehicle (200)

1.4 Authentication Endpoint Tests**POST /api/auth/register**

- Should register a new user and return a JWT token with status 201
- Should return 400 if required fields are missing
- Should return 400 if user with email already exists

POST /api/auth/login

- Should authenticate user and return a JWT token with status 200
- Should return 401 for invalid password
- Should return 401 for non-existent user email

1.5 Public & User Vehicle API Integration Tests**GET /api/vehicles** (Catalog & Filtering)

- Should return all available vehicles with default pagination (200)
- Should filter vehicles by category (200)

GET /api/vehicles/search (Advanced Search)

- Should search vehicles by keyword across make/model (200)

POST /api/vehicles/:id/purchase (User Purchase)

- Should allow an authenticated user to purchase a vehicle and decrease stock (200)
- Should fail if vehicle stock is zero (out of stock) (400/422)

1.6 Cloudinary Live Integration Test

- Should successfully connect to Cloudinary and upload an image (4279 ms)
- Should clean up and delete the test image from Cloudinary (823 ms)

1.7 Code Coverage Summary

File	% Stmts	% Branch	% Funcs	% Lines
All files	87.41	68.57	85.00	88.48
src	94.11	50.00	0.00	94.11
app.ts	94.11	50.00	0.00	94.11
src/config	100.00	100.00	100.00	100.00

File	% Stmts	% Branch	% Funcs	% Lines
cloudinary.ts	100.00	100.00	100.00	100.00
src/controllers	80.58	67.79	100.00	82.35
admin.vehicle.controller.ts	83.54	71.23	100.00	86.95
auth.controller.ts	89.18	100.00	100.00	88.57
user.vehicle.controller.ts	70.37	50.00	100.00	71.42
src/middlewares	88.88	91.66	66.66	88.00
auth.middleware.ts	90.47	91.66	100.00	89.47
upload.middleware.ts	83.33	100.00	0.00	83.33
src/models	100.00	100.00	100.00	100.00
User.ts	100.00	100.00	100.00	100.00
Vehicle.ts	100.00	100.00	100.00	100.00
src/routes	96.87	100.00	0.00	96.87
admin.vehicle.routes.ts	100.00	100.00	100.00	100.00
auth.routes.ts	100.00	100.00	100.00	100.00
health.routes.ts	80.00	100.00	0.00	80.00
user.vehicle.routes.ts	100.00	100.00	100.00	100.00
src/utils	100.00	50.00	100.00	100.00
generateToken.ts	100.00	50.00	100.00	100.00
tests/helpers	100.00	50.00	100.00	100.00
authSetup.ts	100.00	50.00	100.00	100.00
vehicleSetup.ts	100.00	100.00	100.00	100.00

1.8 Backend Test Execution Screenshots

```

sanampreetsingh@fedora:~/Documents/car-dealership-inventory-system/backend$ npm test --coverage
console.log
  > injected env (0) from .env // tip: %suppress logs ( quiet: true )

  at _log (node_modules/dotenv/lib/main.js:131:11)

(node:597669) [MONGODB_WARNING] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.
(Use 'node --trace-warnings ...' to show where the warning was created)
(node:597669) [MONGODB_WARNING] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.
(node:597669) [MONGODB_WARNING] Warning: mongoose: the 'new' option for 'findOneAndUpdate()' and 'findOneAndReplace()' is deprecated. Use 'returnDocument: 'after'' instead.
PASS tests/integration/admin.vehicle.test.ts
Admin Vehicle API Integration Tests
POST /api/admin/vehicles
  ✓ should create a vehicle with an image when admin submits valid data (201) (25 ms)
  ✓ should fail if a vehicle with the exact make, model, year, and category already exists (409) (17 ms)
  ✓ should fail if required fields are missing (400) (28 ms)
  ✓ should block a request missing an auth token (401) (7 ms)
  ✓ should block a standard user from creating a vehicle (403) (16 ms)
PUT /api/admin/vehicles/:id
  ✓ should allow an admin to update text fields only (200) (25 ms)
  ✓ should allow an admin to upload a new image and replace the old one (200) (17 ms)
  ✓ should return 404 if updating a non-existent vehicle ID (18 ms)
  ✓ should return 400 for an invalid MongoDB ID format (8 ms)
  ✓ should block a standard user from updating a vehicle (403) (8 ms)
POST /api/admin/vehicles/:id/restock
  ✓ should allow an admin to increase the quantity (200) (13 ms)
  ✓ should fail if addedQuantity is negative or invalid (400) (8 ms)
  ✓ should block a standard user from restocking a vehicle (403) (7 ms)
DELETE /api/admin/vehicles/:id
  ✓ should block a standard user from deleting a vehicle (403) (8 ms)
  ✓ should return 404 if deleting a non-existent vehicle ID (12 ms)
  ✓ should return 400 for an invalid MongoDB ID format (9 ms)
  ✓ should allow an admin to delete a vehicle (200) (11 ms)

console.log
  > injected env (0) from .env // tip: %custom filepath ( path: '/custom/path/.env' )

  at _log (node_modules/dotenv/lib/main.js:131:11)

PASS tests/integration/auth.test.ts
Auth Endpoints (/api/auth)
POST /api/auth/register
  ✓ should register a new user and return a JWT token with status 201 (122 ms)

```

Figure 1: Terminal output — Admin Vehicle API integration tests passing

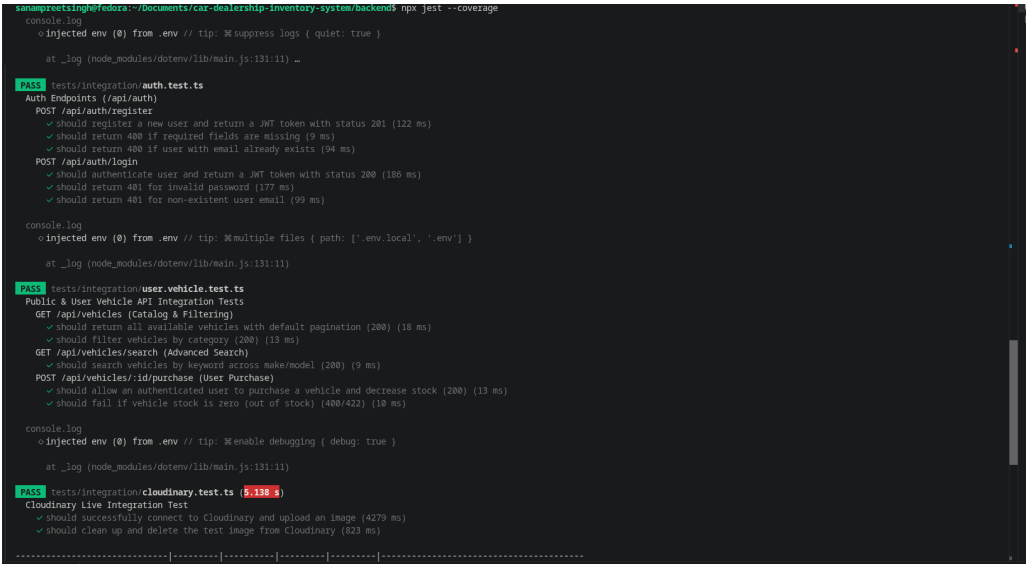


Figure 2: Terminal output — Auth, User Vehicle, and Cloudinary integration tests passing

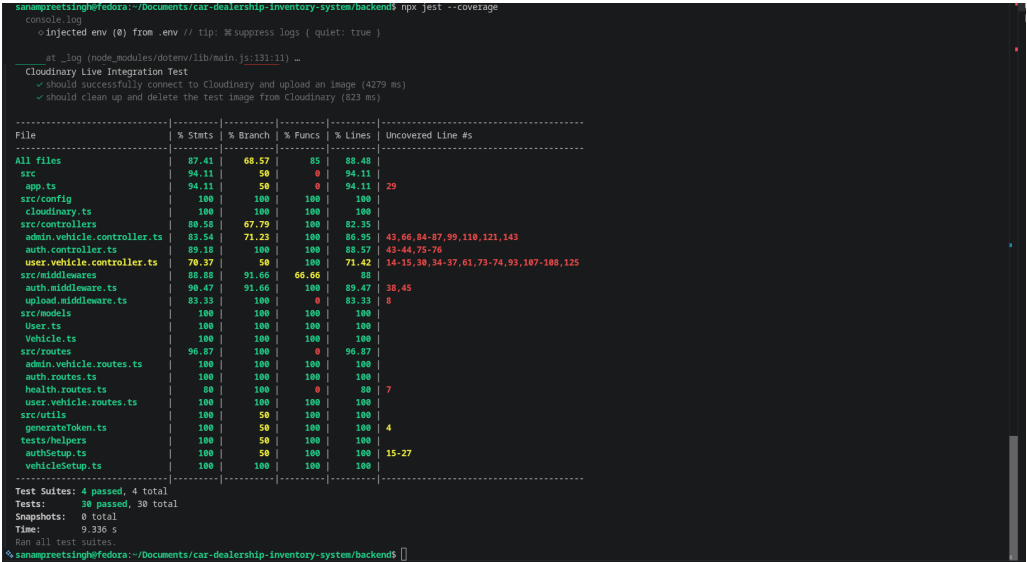


Figure 3: Terminal output — Full coverage summary table and final run summary (30/30 passed)

2 Frontend: Manual Workflow Test Report

2.1 Scope and Methodology

The kata scopes Test-Driven Development explicitly to backend logic. No automated frontend test suite (e.g. Vitest, React Testing Library) was implemented for this project. Instead, the frontend was verified through structured manual end-to-end testing of every core user journey, performed directly against the live backend API. Each test case below records the action taken, the expected result, and the observed outcome, supported by a screenshot.

2.2 Test Case Summary

#	Test Case	Expected Result	Result
1	Load homepage (unauthenticated)	Hero section, stats, and featured vehicles render correctly	PASS
2	Navigate to Login page	Sliding auth panel renders with Sign In form	PASS
3	Toggle to Register form	Panel slides to Create Account form correctly	PASS
4	Fill and submit registration form	New user account created, JWT stored, redirected	PASS
5	Search for a vehicle by keyword with filters visible	Matching results shown with filter sidebar	PASS
6	Click a vehicle card for quick view	Modal opens with full vehicle details and Purchase button	PASS
7	Attempt purchase while logged out	Redirected to Login with return path preserved	PASS
8	Log in as Admin	Navbar reflects Admin role, Admin link visible	PASS
9	Navigate to Admin Dashboard	Inventory stats and vehicle table render correctly	PASS
10	Open "Add New Vehicle" form	Modal form opens with all required fields	PASS
11	Open "Edit Vehicle" form on existing entry	Modal pre-fills existing vehicle data correctly	PASS
12	View updated inventory table after admin actions	Table reflects current stock and edits accurately	PASS

2.3 Detailed Test Cases

Test Case 1 — Homepage Load

Steps: Navigate to the root URL as an unauthenticated user.

Expected: Hero section with call-to-action buttons, trust-signal strip, and featured vehicles grid render without errors.

Result: PASS

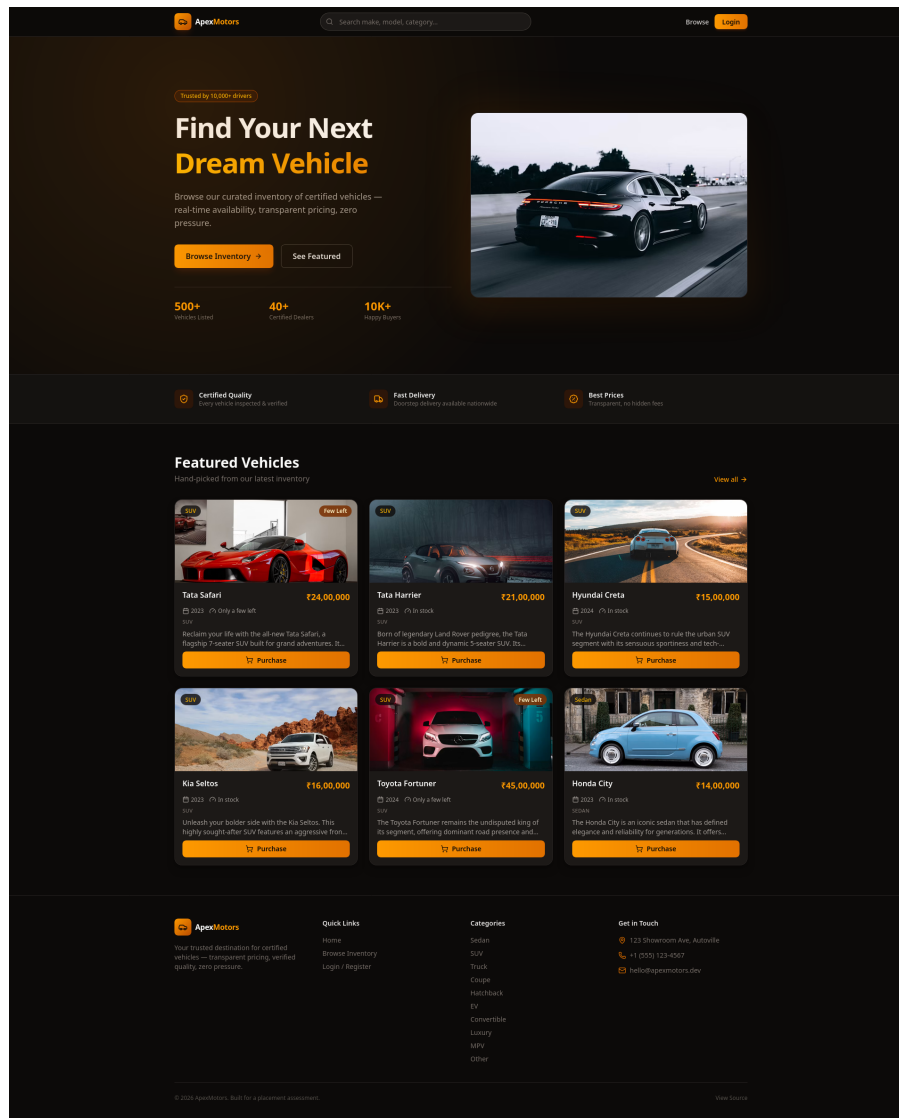


Figure 4: Homepage hero section and featured vehicles

Test Case 2 — Login Page

Steps: Click "Login" in the navbar.

Expected: Sliding auth panel displays with the Sign In form active.

Result: PASS

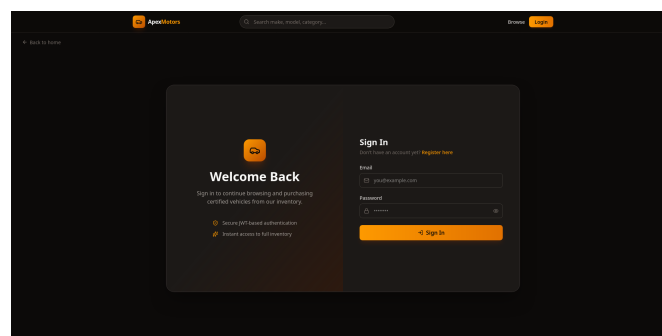


Figure 5: Login (Sign In) form

Test Case 3 & 4 — Register Toggle and Submission

Steps: Click "Register here" to toggle the panel; fill in name, email, and password; submit.

Expected: Panel slides to show the registration form; on submit, a new account is created and the user is logged in automatically.

Result: PASS

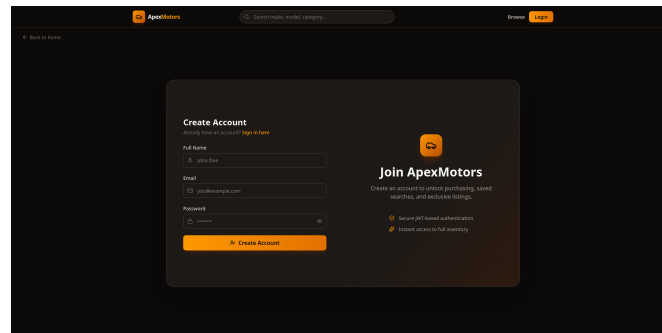


Figure 6: Register form — empty state

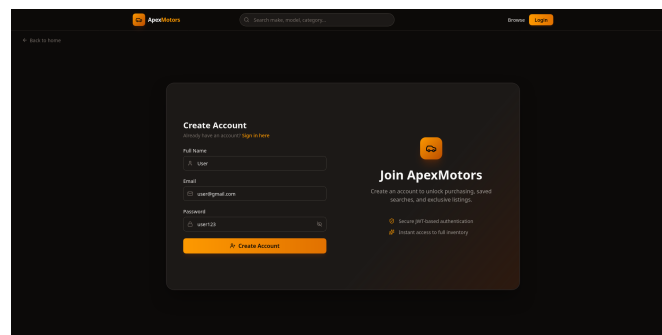


Figure 7: Register form filled with test user data

Test Case 5 — Search with Filters

Steps: Enter a keyword in the global search bar.

Expected: Results page shows matching vehicle(s) alongside the filter sidebar (make, category, price range).

Result: PASS

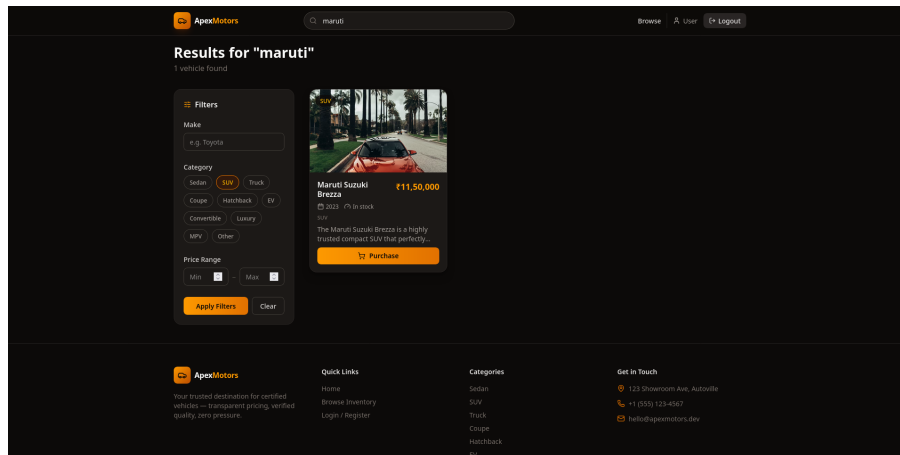


Figure 8: Search results with filter sidebar

Test Case 6 — Quick View Modal

Steps: Click on a vehicle card's image or title.

Expected: A modal opens showing the vehicle's full image, price, specs, and a Purchase button.

Result: PASS

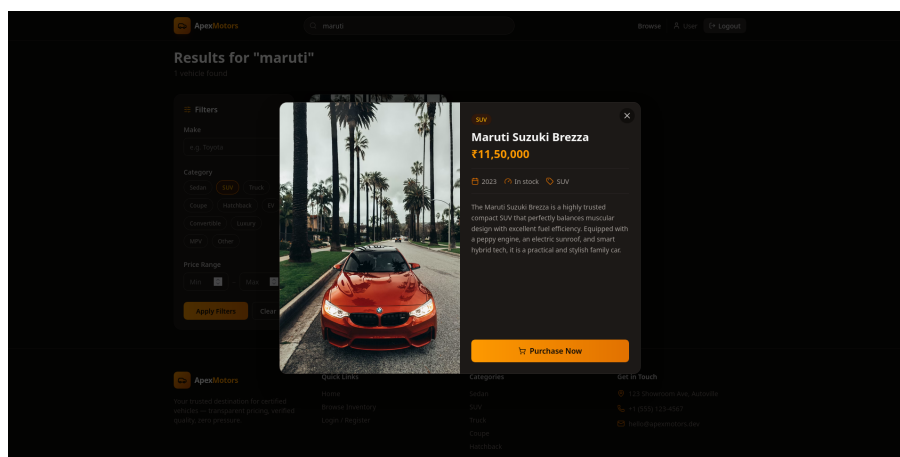


Figure 9: Quick-view modal

Test Case 7 — Purchase Gate for Unauthenticated Users

Steps: While logged out, click "Purchase" on a vehicle.

Expected: User is redirected to the Login page, and after successful login, returned to the original context.

Result: PASS

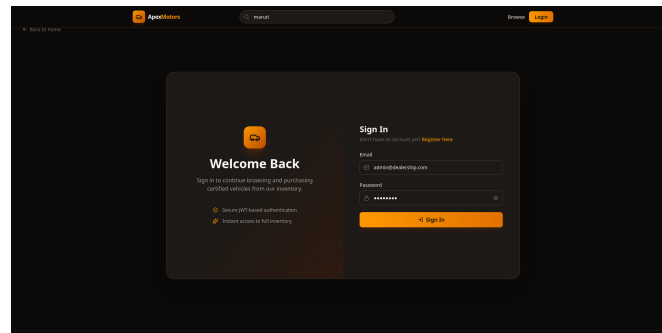


Figure 10: Redirected to Login after attempting purchase while logged out

Test Case 8 — Admin Login and Role-Based UI

Steps: Log in using admin credentials.

Expected: Navbar shows "Admin" link and updates user state to reflect the admin role.

Result: PASS

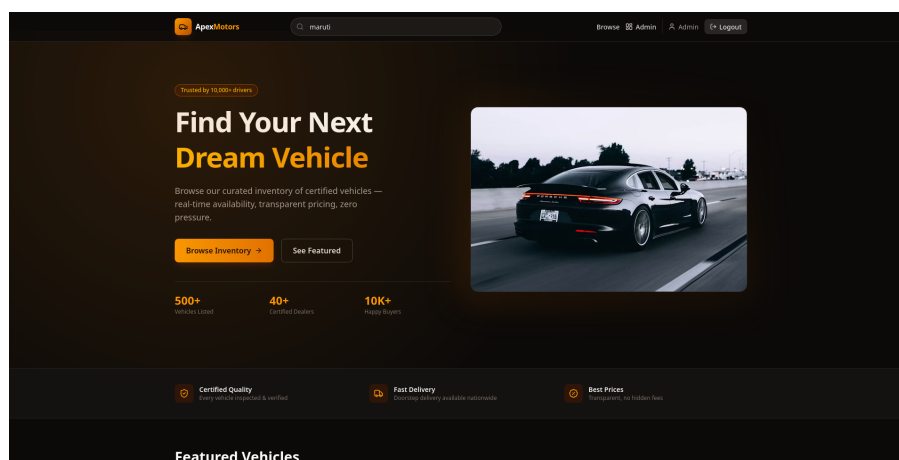


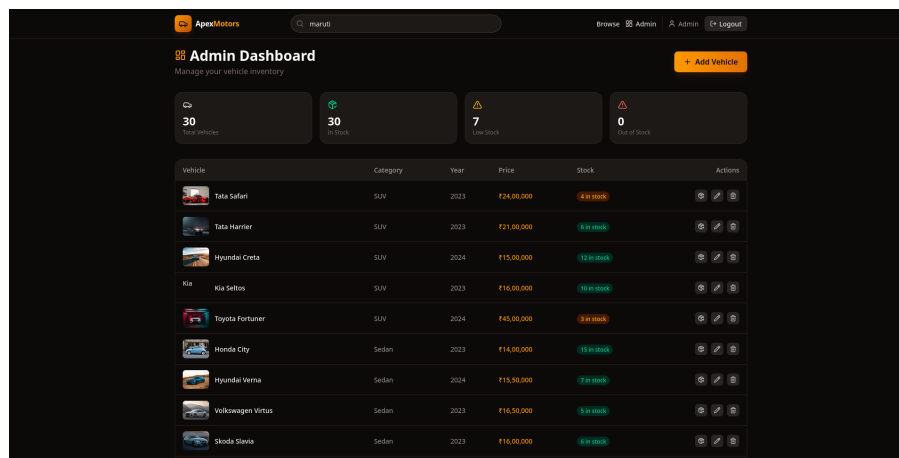
Figure 11: Homepage as logged-in Admin user

Test Case 9 — Admin Dashboard

Steps: Navigate to /admin as an authenticated admin user.

Expected: Dashboard displays live inventory stats and a full vehicle management table.

Result: PASS



The screenshot shows the Admin Dashboard for ApexMotors. At the top, there's a navigation bar with the logo, a search bar, and user controls (Browse, Admin, Logout). Below the header, the dashboard title is 'Admin Dashboard' with a subtitle 'Manage your vehicle inventory' and an '+ Add Vehicle' button. The main content area features four summary cards: 'Total Vehicles' (30), 'In Stock' (30), 'Low Stock' (7), and 'Out of Stock' (0). Below these cards is a table listing vehicles with columns for Vehicle, Category, Year, Price, Stock, and Actions. The table contains 10 rows of vehicle data.

Vehicle	Category	Year	Price	Stock	Actions
Tata Safari	SUV	2023	₹24,00,000	4 in stock	[Edit] [Delete]
Tata Harrier	SUV	2023	₹21,00,000	6 in stock	[Edit] [Delete]
Hyundai Creta	SUV	2024	₹15,00,000	12 in stock	[Edit] [Delete]
Kia Seltos	SUV	2023	₹16,00,000	10 in stock	[Edit] [Delete]
Toyota Fortuner	SUV	2024	₹43,00,000	3 in stock	[Edit] [Delete]
Honda City	Sedan	2023	₹14,00,000	15 in stock	[Edit] [Delete]
Hyundai Verna	Sedan	2024	₹15,50,000	7 in stock	[Edit] [Delete]
Volkswagen Virtus	Sedan	2023	₹16,50,000	5 in stock	[Edit] [Delete]
Skoda Slavia	Sedan	2023	₹16,00,000	4 in stock	[Edit] [Delete]

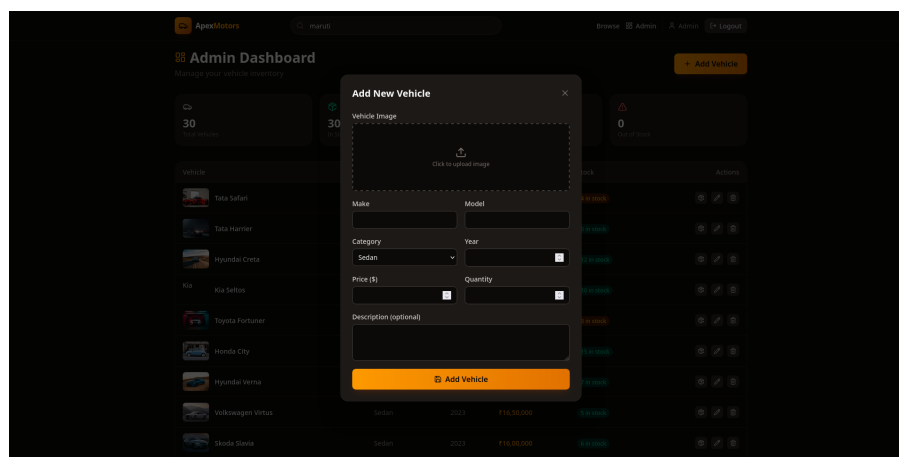
Figure 12: Admin Dashboard with inventory stats and vehicle table

Test Case 10 — Add New Vehicle

Steps: Click "Add Vehicle" on the Admin Dashboard.

Expected: A modal form opens with fields for image, make, model, category, year, price, quantity, and description.

Result: PASS



The screenshot shows the 'Add New Vehicle' modal form overlaid on the Admin Dashboard. The modal has a title bar with a close button. It contains the following fields: 'Vehicle Image' with a placeholder and upload icon; 'Make' and 'Model' text input fields; 'Category' (a dropdown menu currently showing 'Sedan') and 'Year' (a text input field); 'Price (\$)' and 'Quantity' text input fields; and a 'Description (optional)' text area. At the bottom of the modal is an orange 'Add Vehicle' button.

Figure 13: Add New Vehicle modal form

Test Case 11 — Edit Existing Vehicle

Steps: Click the edit icon on an existing vehicle row.

Expected: Modal opens pre-filled with the vehicle's current data, including its uploaded image.

Result: PASS

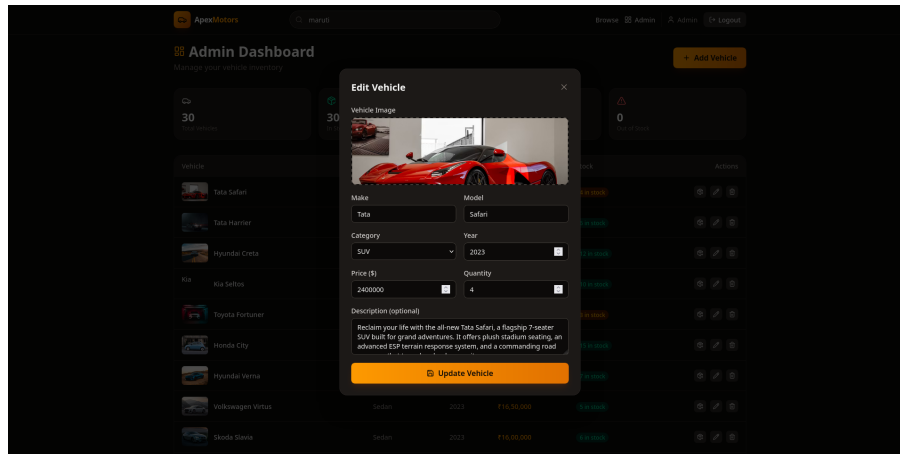


Figure 14: Edit Vehicle modal pre-filled with existing data

Test Case 12 — Inventory Table Consistency

Steps: Return to the Admin Dashboard table after performing add/edit/restock actions.

Expected: Table accurately reflects updated stock levels and vehicle details without requiring a manual page refresh.

Result: PASS

The screenshot shows the 'Admin Dashboard' with a table of vehicles. The table has columns for Vehicle, Category, Year, Price, Stock, and Actions. The data is as follows:

Vehicle	Category	Year	Price	Stock	Actions
Tata Safari	SUV	2023	₹24,00,000	4 in stock	7, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Tata Harrier	SUV	2023	₹21,00,000	6 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Hyundai Creta	SUV	2024	₹15,00,000	12 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Kia					
Kia Seltos	SUV	2023	₹16,00,000	10 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Toyota Fortuner	SUV	2024	₹45,00,000	2 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Honda City	Sedan	2023	₹14,00,000	10 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Hyundai Verna	Sedan	2024	₹15,30,000	7 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Volkswagen Virtus	Sedan	2023	₹16,50,000	5 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
Skoda Slavia	Sedan	2023	₹16,00,000	6 in stock	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100

Figure 15: Admin Dashboard inventory table reflecting live updates

2.4 Frontend Test Summary

Metric	Result
Total Manual Test Cases	12
Passed	12
Failed	0
Testing Method	Manual end-to-end, live backend

3 Conclusion

The backend achieved **30 of 30** passing integration tests across authentication, vehicle catalog, admin CRUD, inventory management, and third-party (Cloudinary) integration, with an overall statement coverage of **87.41%**. The frontend was manually verified end-to-end across all **12** core user workflows defined by the kata requirements, with all test cases passing against the live backend API.